Техническое задание: Telegram-бот для управления домашними делами

Версия: 1.0
Дата: 09.03.2026
Автор: Dima Yatchenko
Назначение: Первая итерация MVP

Введение

Данное техническое задание описывает разработку Telegram-бота для автоматизации учета домашних дел, помощи в приготовлении еды и формирования отчетов о вкладе членов семьи. Бот интегрируется с Perplexity AI для обогащения текста и генерации креативного контента.

Цели проекта

- Автоматизировать учет выполнения домашних задач членами семьи
- Снизить количество конфликтов при распределении обязанностей
- Предоставить умного ассистента для планирования питания
- Мотивировать членов семьи через игровую механику и отчеты
- Обеспечить отказоустойчивое хранение данных

Целевая аудитория

Семьи из 2-6 человек, совместно проживающие в одном доме/квартире, готовые использовать Telegram для координации бытовых вопросов.

Анализ существующих решений

На основе проведенного исследования выявлены следующие популярные решения и их ключевые особенности[1][2][3][4].

Todochi Bot

Функционал:

- Управление повторяющимися задачами (ежедневные, еженедельные, ежемесячные)
- Делегирование задач членам семьи
- Ротация обязанностей
- Напоминания через Telegram
- Совмещение личных и семейных списков

Homi Family Organizer

Расширенные возможности:

- Общий список покупок с защитой от дублирования
- Голосовое добавление задач
- Единый семейный календарь
- Утренний дайджест событий на день
- Поддержка разных часовых поясов
- Отчет о выполненных покупках

Annoying Housemates Bot

Ключевая особенность — случайное распределение задач для честного разделения обязанностей[4]. Статистически выравнивает нагрузку между участниками.

Ledit Bot

Принцип "баланса страха": если кто-то добавляет неприятную задачу, он может получить её сам при случайном распределении[7].

AI Recipe Assistants

Интеграция ИИ для кулинарных задач:

- Распознавание ингредиентов по фото холодильника
- Генерация рецептов из имеющихся продуктов
- Голосовой ввод ингредиентов
- Учет диетических предпочтений
- Пошаговые инструкции приготовления

Примеры: SpiceGuyBot, SmartChef AI[6][9].

Архитектура системы

Технологический стек

Компонент	Технология	Версия
Язык программирования	Python	3.11+
Telegram API	python-telegram-bot	21.x
AI интеграция	Perplexity API	Latest
База данных	SQLite	3.x
Контейнеризация	Docker	24.x
Оркестрация	Docker Compose	2.x

Table 1: Технологический стек проекта

Структура проекта

```
home-assistant-bot/
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── .env.example
├── README.md
├── src/
│   ├── main.py
│   ├── config.py
│   ├── bot/
│   │   ├── __init__.py
```

```
│ │ ├── handlers/
│ │ │ ├── chores_handler.py
│ │ │ ├── cooking_handler.py
│ │ │ ├── reports_handler.py
│ │ │ └── admin_handler.py
│ │ └── keyboards.py
│ ├── database/
│ │ ├── __init__.py
│ │ ├── models.py
│ │ └── repository.py
│ ├── services/
│ │ ├── chores_service.py
│ │ ├── cooking_service.py
│ │ ├── reports_service.py
│ │ └── perplexity_service.py
│ └── utils/
│ ├── logger.py
│ └── helpers.py
├── data/
│ ├── database.db
│ └── backups/
└── tests/
├── test_chores.py
├── test_cooking.py
└── test_reports.py
```

Отказоустойчивое хранилище данных

Для первой итерации используется SQLite с автоматическим резервным копированием.

Механизмы обеспечения отказоустойчивости:

1. **Автоматическое резервирование:**
   - Ежедневные бэкапы базы данных в 03:00 по UTC
   - Хранение последних 30 резервных копий
   - Экспорт бэкапов в отдельную директорию
2. **Транзакционная целостность:**
   - Использование ACID-транзакций SQLite

    - Журнал опережающей записи (WAL mode) для предотвращения блокировок
    - Автоматический rollback при ошибках
3. **Восстановление после сбоев:**
    - Автоматическая проверка целостности БД при старте
    - Восстановление из последнего валидного бэкапа при повреждении
    - Логирование всех операций записи
4. **Docker Volume для данных:**
    - Персистентное хранилище вне контейнера
    - Возможность быстрого восстановления при пересоздании контейнера

Дальнейшая эволюция хранилища:

Для последующих итераций рекомендуется миграция на PostgreSQL с репликацией или использование облачных решений (AWS RDS, Google Cloud SQL).

Функциональные требования

Модуль 1: Учет домашних дел

1.1 Управление задачами

Создание задачи:

- Команда: /addchore или /задача
- Параметры: название, описание (опционально), тип повторения
- Типы задач: разовая, ежедневная, еженедельная, ежемесячная
- Возможность добавления фото (например, фото грязной посуды)

Типовые задачи (пресеты):

- Помыть посуду
- Вынести мусор
- Заказать продукты
- Убрать детские игрушки

- Погулять с собакой/котом
- Полить растения
- Пропылесосить
- Помыть полы
- Сменить постельное белье
- Почистить лоток питомца

Назначение задачи:

- Ручное назначение конкретному члену семьи
- Случайное распределение (режим "справедливая рулетка")
- Ротационное назначение по кругу
- Самостоятельный выбор из списка доступных задач

Отметка выполнения:

- Команда: /done [номер_задачи]
- Быстрая отметка через inline-кнопку
- Опционально: фото результата работы
- Время фиксации выполнения

1.2 Просмотр задач

Команды:

- /mytasks — мои текущие задачи
- /alltasks — все задачи семьи
- /pending — невыполненные задачи
- /history — история за период

Фильтры:

- По исполнителю
- По статусу (выполнено/не выполнено)
- По типу (разовая/повторяющаяся)
- По дате

1.3 Напоминания

- Автоматические напоминания о невыполненных задачах
- Настраиваемое время напоминаний (по умолчанию: 10:00 и 18:00)
- Эскалация: если задача не выполнена 3 дня — уведомление всей семье
- "Мягкие" формулировки от Perplexity AI

Модуль 2: Ассистент повара

2.1 Управление продуктами

Добавление продуктов:

- Команда: /addproduct [название] [количество] [единицы]
- Голосовое добавление через распознавание речи
- Сканирование фото холодильника с AI-распознаванием
- Категоризация продуктов (овощи, мясо, молочное, крупы, специи)

Управление запасами:

- /myproducts — список имеющихся продуктов
- /removeproduct — удаление продукта
- /lowstock — продукты на исходе
- Отметка срока годности с напоминанием

2.2 Генерация рецептов

Основной функционал:

- Команда: /recipe или /рецепт
- Анализ доступных продуктов
- Генерация рецептов через Perplexity AI
- Учет количества продуктов и порций

Параметры генерации:

- Тип блюда: завтрак, обед, ужин, десерт, перекус
- Кухня: русская, итальянская, азиатская, мексиканская и др.
- Время приготовления: быстро (до 30 мин), средне (30-60 мин), долго (60+ мин)

- Сложность: простое, среднее, сложное
- Калорийность: легкое, среднее, сытное

Вывод рецепта:

- Название блюда
- Список ингредиентов с количеством
- Пошаговая инструкция
- Время приготовления
- Калорийность (примерная)
- Фото блюда (генерация через AI или подборка)

2.3 Учет предпочтений (для будущих версий)

- Профили членов семьи с предпочтениями
- Аллергии и непереносимости
- Любимые и нелюбимые продукты
- Диетические ограничения (вегетарианство, веган, кето, безглютеновая)
- История приготовленных блюд
- Рейтинг рецептов от членов семьи

2.4 Список покупок

- Автоматическое формирование списка недостающих продуктов для рецепта
- Команда: /shopping или /покупки
- Добавление произвольных позиций
- Отметка купленного
- Защита от дублирования позиций
- Экспорт списка в текстовый формат

Модуль 3: Отчеты и статистика

3.1 Еженедельные отчеты

Отправка каждое воскресенье в 20:00:

- Общее количество выполненных задач
- Вклад каждого члена семьи (количество и процент)
- Самые популярные задачи недели
- Невыполненные задачи

- Топ-3 самых активных участников

3.2 Ежемесячные отчеты

Отправка 1-го числа каждого месяца:

- Детальная статистика за месяц
- Графики вклада (текстовые гистограммы)
- Сравнение с предыдущим месяцем
- Приготовленные блюда и популярные рецепты

3.3 Шуточные номинации

Генерируются Perplexity AI на основе статистики и выдаются в конце месяца. Примеры номинаций:

- **"Золотая швабра"** — самый активный участник месяца
- **"Железный повар"** — больше всех готовил
- **"Мастер прокрастинации"** — больше всех откладывал задачи
- **"Спаситель игрушек"** — чаще всех убирал детские игрушки
- **"Властелин мусорного ведра"** — лидер по выносу мусора
- **"Посудный маг"** — рекордсмен по мытью посуды
- **"Невидимка месяца"** — меньше всех участвовал
- **"Герой последней минуты"** — выполнял задачи перед дедлайном
- **"Кулинарный экспериментатор"** — пробовал больше новых рецептов
- **"Король/Королева порядка"** — равномерный вклад по всем категориям

Номинации генерируются с юмористическими описаниями от Perplexity AI.

3.4 Команды отчетности

- /stats — личная статистика
- /familystats — статистика всей семьи
- /weekreport — отчет за неделю
- /monthreport — отчет за месяц
- /leaderboard — таблица лидеров

Модуль 4: Административные функции

4.1 Управление семьей

- /addfamily — создание семейной группы
- /addmember — добавление члена семьи (по username)
- /removemember — удаление члена
- /familylist — список членов семьи
- Роли: администратор (создатель), участник

4.2 Настройки

- /settings — меню настроек
- Время напоминаний
- Часовой пояс
- Язык интерфейса (RU/EN)
- Включение/выключение автоматических отчетов
- Режим распределения задач (случайный/ротация/ручной)

4.3 Система мотивации (опционально)

- Начисление баллов за выполненные задачи
- Разные веса для разных задач
- Система достижений (badges)
- Возможность установки семейных целей

Модуль 5: Интеграция с Perplexity AI

5.1 Использование API

Сценарии применения:

1. **Генерация рецептов:**
    - Отправка списка доступных продуктов
    - Получение креативных рецептов
    - Адаптация под кухню и предпочтения
2. **Обогащение текстов отчетов:**
    - Генерация вступительных текстов
    - Создание мотивирующих комментариев
    - Формулирование шуточных номинаций
3. **Умные напоминания:**
    - Генерация вежливых и мягких формулировок

    - Вариативность текстов напоминаний
    - Избегание повторений
4. **Советы по домашнему хозяйству:**
    - Ответы на вопросы типа "как отмыть..."
    - Рекомендации по хранению продуктов
    - Советы по уходу за вещами

5.2 Команды с AI

- /ai [вопрос] — задать вопрос ассистенту
- /suggest — получить рекомендации по блюдам
- /improve [задача] — получить советы по выполнению задачи

Модель данных

Таблица: families

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID семьи
name	TEXT	Название семьи
chat_id	INTEGER UNIQUE	Telegram chat ID
created_at	TIMESTAMP	Дата создания
timezone	TEXT	Часовой пояс (default: UTC)
settings	JSON	Настройки семьи

Table 2: Структура таблицы families

Таблица: members

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID участника
family_id	INTEGER	Внешний ключ на families
user_id	INTEGER	Telegram user ID
username	TEXT	Telegram username
display_name	TEXT	Отображаемое имя
role	TEXT	Роль (admin/member)
joined_at	TIMESTAMP	Дата присоединения
preferences	JSON	Персональные настройки

Table 3: Структура таблицы members

Таблица: chores

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID задачи
family_id	INTEGER	Внешний ключ на families
title	TEXT	Название задачи
description	TEXT	Описание (опционально)
chore_type	TEXT	Тип (one_time/daily/weekly/monthly)
category	TEXT	Категория задачи
assigned_to	INTEGER	ID назначенного участника
created_by	INTEGER	ID создателя
created_at	TIMESTAMP	Дата создания
due_date	TIMESTAMP	Срок выполнения
completed_at	TIMESTAMP	Время выполнения
status	TEXT	Статус (pending/completed/overdue)
photo_url	TEXT	Ссылка на фото (опционально)

Table 4: Структура таблицы chores

Таблица: products

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID продукта
family_id	INTEGER	Внешний ключ на families
name	TEXT	Название продукта
quantity	REAL	Количество
unit	TEXT	Единица измерения
category	TEXT	Категория продукта
expiry_date	DATE	Срок годности (опционально)
added_at	TIMESTAMP	Дата добавления
added_by	INTEGER	ID добавившего

Table 5: Структура таблицы products

Таблица: recipes

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID рецепта
family_id	INTEGER	Внешний ключ на families
title	TEXT	Название блюда
ingredients	JSON	Список ингредиентов
instructions	TEXT	Инструкция приготовления
cuisine_type	TEXT	Тип кухни
cooking_time	INTEGER	Время приготовления (минуты)
difficulty	TEXT	Сложность
calories	INTEGER	Калорийность (ккал)
created_at	TIMESTAMP	Дата создания
cooked_by	INTEGER	ID приготовившего
rating	REAL	Рейтинг (1-5)

Table 6: Структура таблицы recipes

Таблица: statistics

Поле	Тип	Описание
id	INTEGER PRIMARY KEY	Уникальный ID записи
family_id	INTEGER	Внешний ключ на families
member_id	INTEGER	ID участника
period_type	TEXT	Тип периода (week/month)
period_start	DATE	Начало периода
period_end	DATE	Конец периода
chores_completed	INTEGER	Выполнено задач
recipes_cooked	INTEGER	Приготовлено блюд
total_points	INTEGER	Всего баллов
data	JSON	Детальная статистика

Table 7: Структура таблицы statistics

Пользовательские сценарии

Сценарий 1: Добавление и выполнение задачи

1. Пользователь отправляет: /addchore
2. Бот предлагает выбрать из типовых задач или создать новую
3. Пользователь выбирает "Помыть посуду"
4. Бот спрашивает: "Кому назначить задачу?"
5. Варианты: конкретный человек / случайно / ротация / свободная задача
6. Пользователь выбирает "случайно"
7. Бот: "Задача 'Помыть посуду' назначена @username"
8. Уведомление отправляется @username
9. После выполнения @username нажимает кнопку "Выполнено" или отправляет /done
10. Бот подтверждает выполнение и начисляет баллы

Сценарий 2: Генерация рецепта

1. Пользователь отправляет фото холодильника
2. Бот распознает продукты через AI
3. Бот: "Я вижу: курица, картофель, лук, морковь, сметана. Добавить эти продукты?"
4. Пользователь: "Да"
5. Пользователь: /recipe ужин русская кухня быстро
6. Бот обращается к Perplexity AI с контекстом продуктов и параметров
7. Бот отправляет рецепт "Курица в сметане с овощами" с пошаговой инструкцией
8. Пользователь готовит блюдо
9. Пользователь: /cooked — отмечает приготовление
10. Бот сохраняет рецепт в историю и начисляет баллы за готовку

Сценарий 3: Еженедельный отчет

1. Воскресенье, 20:00
2. Бот автоматически генерирует отчет за неделю
3. Обращается к Perplexity AI для создания вступительного текста
4. Формирует статистику по каждому участнику
5. Отправляет сообщение в семейный чат:

 Отчет за неделю 02.03-08.03.2026

Ваша команда выполнила 28 задач на этой неделе!
Это на 12% больше, чем на прошлой. Отличная работа!

 Вклад участников:
• @papa — 12 задач (43%)
• @mama — 10 задач (36%)
• @teen — 6 задач (21%)

 Приготовлено 5 новых блюд
 Осталось 4 невыполненных задачи

Так держать! Вы — лучшая команда! ❤

Нефункциональные требования

Производительность

- Время отклика бота на команды: не более 2 секунд
- Время генерации рецепта через Perplexity AI: не более 10 секунд
- Поддержка до 100 семей одновременно (для MVP)

Надежность

- Доступность системы: 99% (допустимо 7.2 часа downtime в месяц)
- Автоматический перезапуск контейнера при падении
- Резервное копирование БД каждые 24 часа
- Логирование всех критических операций

Безопасность

- Хранение токенов в переменных окружения (.env)
- Не хранить токены в коде или репозитории
- Валидация всех входящих данных
- Ограничение частоты запросов (rate limiting)
- Доступ к данным семьи только её участникам

Масштабируемость

- Модульная архитектура для простого добавления функций
- Готовность к миграции на PostgreSQL
- Возможность горизонтального масштабирования (для будущих версий)

Удобство использования

- Интуитивные команды и меню
- Inline-кнопки для частых действий
- Поддержка голосового ввода
- Мультиязычность (RU/EN для MVP)
- Адаптация текстов под часовой пояс пользователя

Этапы разработки

Фаза 1: Подготовка (1 неделя)

- Настройка окружения разработки
- Создание репозитория
- Настройка Docker-контейнера
- Регистрация бота в BotFather
- Получение API ключей Perplexity
- Проектирование базы данных

Фаза 2: Базовый функционал (2-3 недели)

- Реализация модуля учета домашних дел
- Создание, назначение, выполнение задач
- Базовые команды и обработчики
- Хранилище данных на SQLite
- Простые напоминания
- Механизм резервного копирования

Фаза 3: Ассистент повара (2 недели)

- Управление продуктами
- Интеграция с Perplexity AI
- Генерация рецептов
- Список покупок
- Базовое распознавание продуктов

Фаза 4: Отчеты и статистика (1-2 недели)

- Сбор статистики
- Еженедельные отчеты
- Ежемесячные отчеты
- Генерация номинаций через AI
- Визуализация данных (текстовые графики)

Фаза 5: Тестирование и оптимизация (1 неделя)

- Unit-тесты критических модулей
- Нагрузочное тестирование
- Исправление багов
- Оптимизация запросов к БД
- Улучшение обработки ошибок

Фаза 6: Деплой и запуск (3-5 дней)

- Подготовка production окружения
- Настройка автоматического перезапуска
- Настройка мониторинга и логов
- Пилотное тестирование с реальными пользователями
- Исправление критических проблем

Общая длительность MVP: 7-9 недель

Список команд бота

Основные команды

Команда	Описание
/start	Приветствие и основная информация
/help	Справка по командам
/settings	Настройки бота и семьи

Table 8: Основные команды

Домашние дела

Команда	Описание
/addchore	Добавить новую задачу
/mytasks	Мои текущие задачи
/alltasks	Все задачи семьи
/done	Отметить задачу выполненной
/pending	Невыполненные задачи
/history	История выполнения

Table 9: Команды модуля домашних дел

Кулинария

Команда	Описание
/addproduct	Добавить продукт
/myproducts	Список продуктов
/recipe	Сгенерировать рецепт
/cooked	Отметить приготовление блюда
/shopping	Список покупок
/suggest	Предложить блюдо на сегодня

Table 10: Команды кулинарного модуля

Отчеты

Команда	Описание
/stats	Моя статистика
/familystats	Статистика семьи
/weekreport	Отчет за неделю
/monthreport	Отчет за месяц
/leaderboard	Таблица лидеров

Table 11: Команды отчетности

Администрирование

Команда	Описание
/addfamily	Создать семейную группу
/addmember	Добавить участника
/removemember	Удалить участника
/familylist	Список членов семьи

Table 12: Административные команды

AI-помощник

Команда	Описание
/ai	Задать вопрос AI-ассистенту
/improve	Получить совет по задаче

Table 13: Команды AI-ассистента

Docker-конфигурация

Dockerfile

FROM python:3.11-slim

WORKDIR /app

Установка системных зависимостей

RUN apt-get update && apt-get install -y
gcc
sqlite3
&& rm -rf /var/lib/apt/lists/*

Копирование requirements

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

Копирование кода приложения

COPY src/ ./src/

Создание директорий для данных

RUN mkdir -p /app/data /app/data/backups

Переменные окружения

ENV PYTHONUNBUFFERED=1
ENV TZ=Europe/Moscow

Healthcheck

HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3
CMD python -c "import sqlite3; sqlite3.connect('/app/data/database.db').execute('SELECT 1')"

Запуск приложения

CMD ["python", "-m", "src.main"]

docker-compose.yml

version: '3.8'

services:
home-assistant-bot:
build: .
container_name: home_assistant_bot
restart: unless-stopped
environment:
- TELEGRAM_BOT_TOKEN=
$TELEGRAM_BOT_TOKEN - PERPLEXITY_API_KEY =$
{PERPLEXITY_API_KEY}
- DATABASE_PATH=/app/data/database.db
- LOG_LEVEL=INFO

- BACKUP_ENABLED=true
- BACKUP_HOUR=3
- BACKUP_RETENTION_DAYS=30
volumes:
- ./data:/app/data
- ./logs:/app/logs
logging:
driver: "json-file"
options:
max-size: "10m"
max-file: "3"

.env.example

Telegram Bot Configuration

TELEGRAM_BOT_TOKEN=your_telegram_bot_token_here

Perplexity AI Configuration

PERPLEXITY_API_KEY=your_perplexity_api_key_here

Database Configuration

DATABASE_PATH=/app/data/database.db

Logging

LOG_LEVEL=INFO

Backup Configuration

BACKUP_ENABLED=true
BACKUP_HOUR=3
BACKUP_RETENTION_DAYS=30

Optional: Timezone

TZ=Europe/Moscow

Зависимости Python

requirements.txt

Telegram Bot

python-telegram-bot==21.0.1
python-telegram-bot[job-queue]==21.0.1

HTTP клиент

httpx0.27.0
aiohttp3.9.3

База данных

aiosqlite==0.19.0

AI Integration

openai==1.12.0

Планировщик задач

apscheduler==3.10.4

Работа с датами

python-dateutil2.8.2
pytz2024.1

Логирование

loguru==0.7.2

Валидация данных

pydantic2.6.1
pydantic-settings2.2.1

Работа с изображениями

Pillow==10.2.0

Утилиты

python-dotenv==1.0.1

Мониторинг и логирование

Логирование

- Использование библиотеки loguru
- Уровни логов: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Ротация логов: максимум 10MB на файл, хранение 7 дней
- Отдельные логи для разных модулей

Структура логов:

```
logs/
├── app.log # Основной лог приложения
├── database.log # Операции с БД
├── telegram.log # Telegram API взаимодействия
```

```
├── perplexity.log # Запросы к Perplexity AI
└── errors.log # Только ошибки
```

Мониторинг

Для production рекомендуется:

- Prometheus + Grafana для метрик
- Sentry для отслеживания ошибок
- Uptimerobot для проверки доступности
- Алерты в Telegram при критических ошибках

Расширения для будущих версий

Версия 2.0

- Интеграция с календарями (Google Calendar, Яндекс.Календарь)
- Веб-интерфейс для просмотра статистики
- Экспорт отчетов в PDF
- Голосовые команды (распознавание речи)
- Фото-отчеты о выполнении задач
- Интеграция с умным домом (Home Assistant)

Версия 3.0

- Мобильное приложение (React Native)
- Миграция на PostgreSQL с репликацией
- Поддержка множественных семей на одного пользователя
- Marketplace рецептов от сообщества
- Геймификация: достижения, челленджи, награды
- Интеграция с доставкой продуктов (API супермаркетов)
- AI-прогнозирование потребностей семьи
- Распознавание речи для голосового управления

Версия 4.0

- Компьютерное зрение для проверки выполнения задач
- Умные уведомления на базе ML (оптимальное время)
- Интеграция с финансами (учет затрат на продукты)
- Социальная сеть семей для обмена опытом

- Персонализированные рекомендации на основе истории
- Режим коллаборации с другими семьями (соревнования)

Риски и их митигация

Риск	Вероятность	Митигация
Ограничения API Telegram	Средняя	Rate limiting, очереди
Превышение квоты Perplexity	Средняя	Кэширование, лимиты
Потеря данных	Низкая	Резервное копирование
Перегрузка сервера	Низкая	Мониторинг, масштабирование
Неадекватное использование	Низкая	Модерация, rate limiting

Table 14: Матрица рисков

Критерии приемки MVP

- Бот успешно запускается в Docker-контейнере
- Реализованы все основные команды модуля домашних дел
- Работает интеграция с Perplexity AI для генерации рецептов
- Генерируются еженедельные и ежемесячные отчеты
- Система резервного копирования функционирует корректно
- Бот обрабатывает минимум 10 команд в секунду без ошибок
- Покрытие кода тестами минимум 60%
- Документация README с инструкциями по развертыванию
- Успешное тестирование с реальной семьей (минимум 3 участника)
- Отсутствие критических багов после недели использования

Заключение

Данный Telegram-бот представляет собой комплексное решение для автоматизации домашних дел и помощи в планировании питания. Использование Python, Docker и интеграции с Perplexity AI обеспечивает гибкость, надежность и возможность дальнейшего масштабирования.

Модульная архитектура позволяет постепенно развивать функционал, добавляя новые возможности без переработки базовой системы. Отказоустойчивое хранилище на базе SQLite с автоматическим резервным копированием обеспечивает сохранность данных для первой итерации, с возможностью миграции на более мощные решения в будущем.

Геймификация через отчеты и шуточные номинации повышает вовлеченность членов семьи и снижает психологическое сопротивление выполнению бытовых задач. AI-ассистент для приготовления еды решает вечную проблему "что приготовить из того, что есть", экономя время и снижая пищевые отходы.

References

[1] Bugogish. (2025). Telegram bot to manage co-living (family) and personal tasks and reminders. Reddit /r/TelegramBots. https://www.reddit.com/r/TelegramBots/comments/1lgnv0s/telegram_bot_to_manage_coliving_family_and/

[2] Telegram.org.ru. (2025). Todochi - менеджер личных и семейных задач с напоминаниями. https://telegram.org.ru/29405--todochi-menedzher-lichnyh-i-semeynyh-zadach-s-napominaniyami.html

[3] Beyers, R. (2025). Build an AI Recipe Bot with Telegram + Make + ChatGPT. YouTube. https://www.youtube.com/watch?v=wFpeGZU3TC0

[4] DevPost. (2018). Annoying-Housemates-Bot: A Telegram bot to automate organizing house chores. https://devpost.com/software/annoying-housemates-bot

[5] Homi Family. (2024). Homi — семейный органайзер в Telegram. https://www.homifamily.ru

[6] SpiceGuyBot. (2025). Free AI Recipe Assistant | Smart Cooking. https://spiceguybot.com

[7] JanF93. (2022). Fair todo assignments in telegram group chats. Reddit /r/TelegramBots. https://www.reddit.com/r/TelegramBots/comments/sykhmg/fair_todo_assignments_in_telegram_group_chats/

[8] Work24.ru. (2026). Лучшие Telegram боты для задач: работа и личная жизнь. https://work24.ru/journal/193-chat-boty-v-telegram-kotorye-realno-uproshchayut-jizn-rabota-dela-otdyh

[9] SmartChef AI. (2025). #1 AI Kitchen Assistant | Revolutionary Food Tech. https://smartchefmobile.com

[10] Laukkarinen, R. (2022). Telegram bot for the house chores for my children. GitHub. https://github.com/ronilaukkarinen/chores-telegram-bot